



Princess Sumaya جامعة
University الأميرة سميرة
for Technology للتكنولوجيا

Design and Implementation of IoT-Based Smart Mobile Animal Housing System

By

Yara Rahhal, Batool Allan, & Aya Almasri

Supervised by

Dr. Rami Wahsheh

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE

In

Communications Engineering/IoT

at

PRINCESS SUMAYA UNIVERSITY FOR TECHNOLOGY

Amman, Jordan

Second Semester 2025-2026

This is to certify that I have examined
this copy of an engineering documentation by

Yara Rahhal, Batool Allan, & Aya Almasri

And have found that it is complete and satisfactory in all respects,
And that any and all revisions required by the final Examining Committee have been made



Dr. Rami Wahsheh (Supervisor)

Dr. Luai Al-Tarawneh (Department Head)

Acknowledgments

We would like to express our gratitude to Dr. Rami Wahsheh, our supervisor, for being supportive, helpful and for guiding us throughout the project. He was the reference we went back to when we needed anything. We would also like to thank our friends and family, not only for being there for us the moment we started this chapter in our lives, but for also giving us the support we needed when things got tough.

Generative AI Usage Declaration

We, Yara, Batool, and Aya hereby declare that:

- I. We are solely responsible for the content provided in this work, including any AI-generated content.
- II. We are solely responsible for any copyright issues that could result from using generative AI tools or otherwise.
- III. All the Generative AI tools we have used during the preparation of this work are listed in the table below, and failure to disclose any usage is considered academic misconduct or plagiarism that could result in degree revocation.

Tool (Name & Version)	Extent of use (Chapter(s), Section(s) or Page(s))	Purpose of Use (Code, Figures, Proofreading...etc.)
ChatGPT (OpenAI GPT-5.5)	Chapter 3 and Chapter 4 figures	Assistance in generating and refining system architecture diagrams and workflow figures
ChatGPT (OpenAI GPT-5.5)	Various chapters and sections throughout the report	Proofreading, grammar correction, paragraph restructuring, and academic writing improvements
ChatGPT (OpenAI GPT-5.5)	Testing and Results sections	Assistance in improving technical descriptions and formatting

Signature: Yara Rahhal, Batool Allan, Aya Almasri

Date: May 12, 2026

Abstract

Internet of Things (IoT) is a new rapid growing technology that connects everything to the Internet reducing manual intervention. Sensors collect data from the environment, transmit it wirelessly to the internet, and then the data is processed. After that, actuators act based on the analysis. It is used in many fields such as smart homes, wearables, such as smartwatches, and healthcare. However, many IoT devices have weak security protocols, making them vulnerable to cyberattacks.

This project designs a smart mobile animal housing system that consists of many sensors, including a temperature sensor, a rain detector, a water level sensor, a light intensity sensor, a GPS module, and a motion detection sensor, that will collect data from the environment. Then the data is processed using the ESP32-S3 microcontroller, which makes the decisions based on the collected data. Then actuators such as water pump, relay, LED, fans, and H-bridges, act based on the microcontroller's decision.

The core innovation in this project is that it is floorless, which results in reducing waste accumulation. It moves forward or turns right using wheels based on the owner's decision. The user can turn on/off the actuator using the Blynk dashboard, where all the actuators and readings of the sensors are displayed. The data is visualized using Grafana and it is also stored in the InfluxDB database.

Keywords: Internet of Things (IoT), Smart Animal Housing, Automated Environmental Control, Remote Monitoring

Table of Contents

List of Tables	2
List of Figures	3
1 Introduction	1
1.1 Motivation.....	2
1.2 Objectives	3
1.3 List of Design Requirements	3
1.4 List of Design Constraints.....	5
1.5 List of Engineering Standards.....	5
1.6 Design Overview	5
1.7 Load Distribution	7
2 Background and Literature Review	8
2.1 Background	8
2.2 Literature Review	9
2.3 Limitations	10
2.4 Summary.....	11
3 Design	12
3.1 Analysis of Design Requirements.....	12
3.2 Analysis of Design Constraints	13
3.3 Discussion of Engineering Standards	14
3.4 Different Design Approaches.....	15
3.5 Design Development	16
4 Results	26
4.1 Prototype Setup.....	26
Figure 4-1: System Workflow of Blynk, InfluxDB, and Grafana.	31
4.2 Testing	31
4.3 Discussion of Results	32
4.4 Validation of Design Requirements within the Realistic Constraints	34
5 Conclusions and Future Work	36
References	38
Appendices	39
Appendix A: Offering Template	40
Appendix B: Code	42
ESP32 Code 1:	42
ESP32 Code 2:	54

List of Tables

Table 1-1: Load Distribution.	7
Table 3-1: System Components and Prices.	25
Table 4-1: First ESP32 Components.	27
Table 4-2: Second ESP32 Components.	28
Figure 4-1: System Workflow of Blynk, InfluxDB, and Grafana.	31
ESP32 Code 1:	42
ESP32 Code 2:	54

List of Figures

Figure 1-1:Overall Design	7
Figure 3-1: First ESP32 Architecture.	19
Figure 3-2: Second ESP32 Architecture.	20
Figure 3-3: Overall Architecture	21
Figure 3-4: Environmental Control Subsystem	21
Figure 3-5: Resource Management Subsystem	22
Figure 3-6: Security and Intrusion Detection Subsystem	23
Figure 3-7: Power Management	25
Figure 4-1: System Workflow of Blynk, InfluxDB, and Grafana	32
Figure 4-2: Blynk Dashboard	35

1 Introduction

Animals in shelters often live in unsafe and harsh conditions, leading to several health, behavioral, and developmental problems. The animals living in these traditional shelters suffer due to the lack of care and supervision. These shelters are often overcrowded which leads to disease spread, poor air quality, and conditions that threaten the animals' welfare. Animals need continuous monitoring and supervision which includes feeding, re-filling water tanks, air ventilation control, and maintaining a clean comfortable environment.

In traditional shelters, the caregiver usually works for 8 hours per day, which equals almost 40 hours per week, and that is time and energy consuming. The caregiver should feed, clean, groom, and observe symptoms of illness every single day. This can be mentally and physically exhausting as it requires lifting, long hours of standing, and continuous movements. If the owner has a large farm or shelter, the workload becomes almost unbearable. The Internet of Things (IoT) became a solution for these problems. It reduces labor, monitoring, and ensures consistency even if human presence is not available.

Traditional shelters are fixed in one location, this results in overgrazing and waste builds up. This also reduces land productivity and limits animals' access to fresh grass. They also lack security, which increases the risk of theft, unauthorized access, and animal escapes.

IoT has solved all of these problems. IoT is an a network of interconnected devices that collect and exchange data. It offers real-time monitoring and automated control. It provides solutions to farmers who want to improve the living conditions of animals on farms and make it easier to control, manage, and provide food, water, and other necessities. This proposed system integrates sensors and actuators to monitor and adjust different environmental and operational parameters.

This paper represents the design and implementation of an IoT-Based Smart Mobile Animal Housing System. The main objective of this project is to develop a mobile housing unit equipped with sensors and actuators. The system deploys two ESP32 microcontrollers as the central processing units that collect, process, and store data collected from the sensors.

The first ESP32-S3 is responsible for environmental monitoring and control, such as temperature reading, rain detection, light control, and motion detection.

The second ESP32-S3 controls water level monitoring, water pump control, door locking/unlocking based on a 4-digit code that is randomly generated which is entered on the keypad, wheel movement, and GPS tracking.

This project uses solar power to reduce the power consumption. It requires a renewable energy source to reduce the dependency on external power sources.

1.1 Motivation

Many herbivores raised on farms, such as cattle, sheep, goats, and horses, primarily rely on grass as their main source of nutrition. Over time, continuous grazing leads to overgrazed and damaged pasture-land. Additionally, waste accumulates within the housing and feeding areas, creating unhygienic conditions that can spread diseases.

Since most grazing animals defecate near their feeding zones, improper waste management can result in contamination of both food and water sources. Waste accumulation increases the risk of infections and parasites. Most existing animal shelters are vulnerable to theft, intrusion, or animal escape due to insufficient security controls.

Animal Shelters conditions are affected by many factors, such as rainfall and temperature shifts. Too much sunlight is extremely harmful to animals, and this may lead to heat stress, dehydration, skin irritation, and even skin cancer, while poor ventilation and high humidity lead to disease spreading.

According to research conducted in 2009, “An animal that is well fed and watered, as well as being in good condition, will have high resistance to disease. Good management can do much to remove or reduce the effects of adverse environmental factors, such as climatic stress, which would otherwise weaken the body’s natural defenses” [1].

This study mentions several environmental factors that affect animal health, comfort, and productivity. These factors include light, dust, and toxic gases. Direct sunlight affects animal performance, breeding, and egg production. Since most traditional shelters are indoor, artificial light is used to regulate laying cycles and feeding behavior.

Another important factor that influences animal health and productivity is body temperature regulation. All livestock animals are homeothermic, meaning they maintain a nearly constant internal body temperature regardless of external environmental conditions, usually within 1–2°C [1]. Poultry do not have sweat glands, so all heat leaves the body through the respiratory system.

In hot and humid climates, moisture cannot be absorbed efficiently by the air, resulting in additional heat stress.

In addition, continuous use of the same location causes long term soil degradation and reduces the sustainability of farm resources. Traditional animal housing management increases operational costs and limits productivity in large-scale farms.

1.2 Objectives

The main objectives of this project are:

- Building a smart, mobile, floorless animal housing system that relocates daily to reduce waste accumulation and provide the animals access to fresh grass.
- Integrating solar panels to reduce power consumption
- Automating environmental control to provide the animals the perfect environment.
- Developing a smart water management system
- Improving security by granting access to authorized users only
- Monitoring the housing system by installing a GPS module
- Real-time monitoring and controlling the housing system remotely using Blynk
- Storing data in InnfluxDB and visualizing it using Grafana

1.3 List of Design Requirements

1. Data Collection

Sensors continuously gather data from the animal house environment. Motion and intrusion sensors detect human presence and unauthorized access. Sensor readings are taken at regular intervals.

2. Data Transmission

Sensor data is transmitted via Wi-Fi using the ESP32 microcontroller to the Blynk. Data packets are timestamped for synchronization and logged into a database. The system uses HTTP communication for data transmission between the ESP32 and the database server.

3. Data Storage and Time Stamping

All collected sensor data shall be stored in InfluxDB with accurate timestamps for environmental monitoring, behavioral analysis, and system performance evaluation. The collected data, such as temperature, rain detection, water level, GPS location, and actuators operation, such as fans activity, water pump, and door access. The data is visualized using Grafana dashboards to analyze and optimize it.

4. Automated and Control Response

The fan automatically activates when the temperature exceeds a certain threshold. The water pump refills the tank when the water is not detected. The housing structure relocates daily. The smart door system unlocks after the correct password entry and triggers an alarm upon failed attempts or intrusion detection.

5. Alerts and Notifications

Notifications are sent to the user in case of:

- Intrusion Detection
- Rain detection
- Abnormal Temperature

6. User Interaction and Monitoring

By using a mobile or web application, the owner will be provided with real-time monitoring and he can control different parameters. A map with the location's coordinates will be displayed to track the housing system.

7. Data Analysis for Optimization

Environmental trends and energy usage are identified by time-series data analytics. Daily statistics using Grafana are visualized on dashboards.

8. Dual Microcontroller Coordination

The system used 2 microcontrollers so that the load is distributed so that process overload is prevented and making maintenance and scalability easier.

9. Remote Manual Override

The owner is able to manually control the actuators using Blynk.

10. Security Logging

Every single access attempt is recorded with timestamps to monitor security events.

1.4 List of Design Constraints

- The total cost should not exceed 500 JODs
- The system should operate with minimum power consumption to ensure energy efficiency and environmental sustainability.
- The system should be scalable.

1.5 List of Engineering Standards

The following standards are used in this project:

1. WiFi IEEE 802.11: this standard is used for wireless communication between the microcontrollers and the Blynk platform and the InfluxDB server, allowing:
 - Real-time monitoring
 - Alerts and notifications sendings
 - Transmitting data to cloud
 - Visualizing the dashboard
2. GPS: this standard is used to track the location of the housing system, allowing:
 - Tracking of the mobile animal housing system
 - Monitoring the movement of the system
 - Displaying the location of the system on Blynk
3. I2C communication protocol: this standard is used for the communication between the ESP32s and the LCD
4. UART serial communication: this standard is used for the communication with the GPS module to receive the latitude and longitude and to display the location of the housing system on Blynk.

1.6 Design Overview

The proposed system introduces a new concept, the integration of mobility into animal housing. By integrating wheels and motorized movement, the housing system can move regularly to new

grazing areas. This prevents waste accumulation and lowers the risk of disease spreading. As a result, animal hygiene and living conditions are improved.

The system also uses renewable solar energy to address energy consumption, making the system energy efficient.

In addition, the system uses two ESP32 microcontrollers to distribute the tasks and prevent process overload. These microcontrollers manages multiple subsystems including:

- Environmental Control system consists of a temperature monitoring, fan automation, rain detection, light intensity and LED control.
- Resource Management System consists of water level monitoring, automatic water pump control, GPS tracking, and mobility management.
- Security and Intrusions Detection System includes a keypad, a servo motor to lock/unlock the door, a PIR motion detection sensor, and a buzzer.

Every single subsystem has its own sensors and actuators to perform automated tasks when needed. But the owner can manually override these decisions using the Blynk's dashboard for remote control and monitoring.

All of the sensor readings are stored in a cloud-based management system for real-time monitoring and analysis.

WiFi is the main communication method used for communication between the microcontrollers and the cloud platforms. The overall architecture of the system is designed to improve animal welfare, reduce manual labor, and provide a scalable solution for smart farming applications.

Figure 1- 1 shows the overall architecture of the system.

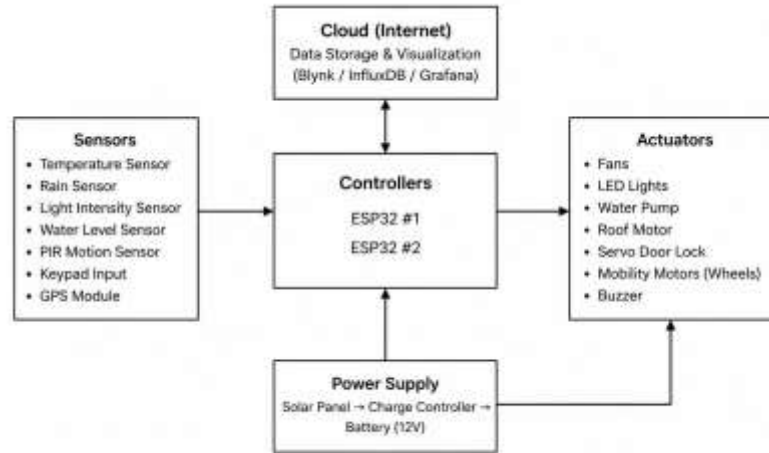


Figure 1-1:Overall Design.

1.7 Load Distribution

Table 1-1 shows how the load was distributed among team members

Table 1-1: Load Distribution.

Task	Yara	Aya	Batool	Month
Introduction and Problem Definition	25%	35%	40%	October
Literature Review	15%	30%	55%	November
System Design and Sketching	15%	30%	45%	November
Hardware Selection and Circuit Design	25%	30%	45%	December
ESP32 programming and sensor integration	50%	25%	25%	January
Blynk integration	50%	30%	20%	January
InfluxDB and Grafana setup	50%	25%	25%	February
Prototype assembly and testing	35%	35%	30%	February
System validation and performance testing	35%	30%	35%	April
Final report and presentation preparation	30%	35%	35%	May

2 Background and Literature Review

2.1 Background

The living conditions of the animals are the prime factory of their productivity. Traditional animal shelters suffer from waste buildups, poor ventilations, and lack of continuous monitoring. All of these lead to illness and disease spread.

One of the main problems that face traditional animals is their static structure. Since it stays in the same place for long periods of time, the animals graze in the same area which leads to overgrazed fields and damaged pasturelands. Waste accumulates quickly as well, which increases the risk of infections and parasites. These shelters are extremely vulnerable to extreme weather conditions such as extreme temperatures, rainfall, poor ventilation, and direct sunlight exposure which may lead to skin cancer. Another vulnerability of traditional shelters is security such as unauthorized access and security breaches.

This project provides the solution to all of the previous problems by integrating the Internet of Things (IoT) to manage environmental control, improve security, and provide animals with fresh grass through mobility. This enables the housing system to move to minimize the waste accumulation and improve hygiene. The ESP32, which is powered by the solar panels, is the brain of the whole system, which gathers data from the sensors, makes decisions, then sends commands to the actuators to act based on the collected data.

The system also includes a door that unlocks if a 4-digit randomly-generated code is entered on the keypad, and sends an alert if the code is incorrect.

All of the collected data is then stored in InfluxDB with timestamps and is then visualized using Grafana.

A smart animal housing system is created by combining mobility, automation, renewable energy, and real-time monitoring. This smart housing system will improve the animals' welfare, reduce manual labor, and will increase operational efficiency.

2.2 Literature Review

Previous studies were needed before starting this project to build a strong foundation. These previous studies used IoT in smart agriculture, animal monitoring, and smart animal housing systems.

“A Smart Monitoring System for a Future Smarter Dairy Farming” [2] study built a smart monitoring system for dairy farming. This system included sensors that collect data from the environment and actuators acted based on these data. The study focused on environmental monitoring, data-driven decision-making, and improving animal welfare. This study is related to the current project because it talks about the importance of real-time monitoring using sensors and automated responses using actuators in animal housing systems.

“Smart Pet House” [3] study built a smart pet house which enabled automatic feeding, temperature control, waste cleaning, and air purification. This study proved that IoT can be used to improve the animal’s lifestyle and make it more comfortable as it also reduced manual intervention. It used a mobile application for real-time monitoring and remote control.

“IoT-Smart Monitoring of Pet Housing” [4] study built an IoT-based smart monitoring system for pet housing. Microcontrollers and sensors to collect data were used for environmental and feeding monitoring. Different sensors were used such as temperature, humidity, and feeding level. Cloud platforms were used to store the data collected from the sensors. This study is actually very beneficial for the current project because it proved that IoT can be used to combine environmental sensing, cloud connectivity, mobile access, and automated control to improve animal care.

“Smart Pet House” [5] study built a basic smart pet house system. It proved that using embedded systems and sensors to control animal housing conditions is practical. But the system depended solely on fixed power supplies and was designed for indoor environments only. This limits installing it outdoors, which is what this project aims for.

“Design of Internet of Things for Smart Residence” [6] study proposed an IoT-based architecture to remodel a traditional housing system into a smart one. It included smart lighting, environmental monitoring, and remote control via a user interface. One of the main objectives of this study was to design a low-cost efficient IoT system. Sensors, actuators, microcontrollers, and a gateway were integrated to monitor and control home appliances remotely.

“Smart Poultry Farming 4.0: Leveraging IoT for Climate Control and Productivity” [7] study’s main objective was to improve poultry farming. It deployed sensors to monitor the environment, then the collected data was sent to a microcontroller so that it can be processed, then the microcontroller sends the actuators demands on how to act. It is designed to be scalable across large and small farms.

All of the previous studies proved that IoT is the perfect solution to reduce manual labor. They show that sensors, actuators, microcontrollers, cloud platforms, and mobile applications can work together perfectly. But almost all of the previous studies focused on indoor animal housing, while this project focuses on a mobile, solar powered, floorless housing system that improves animal welfare, reduce waste accumulation, reduce power consumption, and provide access to fresh grass.

2.3 Limitations

Early studies offered great solutions for smart animal housing, but limitations were still there. Many IoT systems are based on continuous Wi-Fi communication, active sensing, or even high-power actuators, and these make them unsuitable for long-term use outdoors. Others are powered by mains electricity, keeping them from roaming and enabling them to function in the remote farms where the electricity is low. The other limitation is mobility. Existing systems are mainly built as fixed smart pet houses or stationary livestock monitoring units. They are not solutions for the issue of overgrazing or wastes accumulated when animals are kept in the same spot the same amount of time. In contrast, the proposed system, however, brings a mobile, floor-less structure that can navigate day to day, introduce animals to healthy grass and lower manure concentration. Another drawback is that this is a common limitation with so many prototypes testing only in indoor or controlled environments. They do not take in full account of outdoor farm conditions like rain, dust, sun, mud, and changing temperatures. This results in very poor generalizability in actual agriculture settings. Security is similarly poor in many of the older systems. Although there is monitoring and alerts provided by some studies, in some cases, they do not always contain robust access control mechanisms. The current project solves this problem by installing a smart door that is connected to a keypad, and if the entered code is incorrect, then a buzzer would be turned on. Most of the previous studies were solely based on threshold-based automation, but they did not focus on long-term data analysis. The current project solves this problem by storing the collected

data in InfluxDB with timestamps and real-time reading and visualization of data with Grafana dashboards.

2.4 Summary

Internet of Things (IoT) technologies in the reviewed literature provide effective technology to enhance animal housing systems with automation, environmental monitoring, and remote management. There have been successful implementations of smart pet houses and livestock monitoring systems with sensors, actuators, microcontrollers, mobile applications, and cloud platforms to improve animal welfare and minimize manual effort. The literature review describes that temperature monitoring, automatic ventilation, water management, feeding systems, and intrusion detection are the main features in smart animal housing. It also underlines the importance of real-time observation, data logging, and mobile accessibility for greater operational efficiency and supporting fast decision-making by the animal caretakers. But, most of the current systems were built with the fixed indoor environment, and use dependency highly reliant on external power sources, or components that consume major amounts of energy, making them relatively limited in the context of agricultural application on the outdoor farm. Also, most of these systems lack mobility, sophisticated security mechanisms, and long-term data analysis. The proposed work builds on those previous studies by introducing a mobile, floor-less animal housing system that is powered by solar energy. Integrating ESP32 microcontrollers, GPS location, keypad-based locks, security system, automated environmental control, a water level monitoring, Blynk portable monitoring, InfluxDB cloud storage, and Grafana visualization has a more efficient and larger scale alternative to actual farm life. Hence, this study advances current literature in the context of the interdisciplinary integration of mobility, renewable energy, security, automation, and data-based monitoring in an integrated smart animal housing system that enhances animal welfare, minimizes the generation of waste, and improves farm efficiency.

3 Design

3.1 Analysis of Design Requirements

These design requirements are the foundational technical specifications that outline the system architecture translating to the proposed project objectives. Each requirement is a direct response to a major problem of standard animal accommodation discovered within the motivation, such as inadequate environmental control, waste accumulation, limited security, and absence of continuous monitoring. These requirements influence the choice of hardware, software logic, and subsystem integration. The environmental control requirements include automatic fans turned on when the temperature exceeds a certain threshold and the roof closing when it rains, inferred from the physiological needs of livestock animals. As established in the background, animals such as poultry lack sweat glands and are particularly susceptible to heat stress, while poor ventilation and high humidity facilitate the spread of disease. Therefore, these requirements demand rapid responsiveness via real-time sensor-actuator loops. The selection of the LM35 temperature sensor and the YL-83 rain sensor was based on linear output, reliability in outdoor conditions, simple integration, and low power consumption—meeting the system’s energy-efficiency goals. Refilling of automatic water tanks and the daily structural relocation solve the issues of waste accumulation and overgrazing. The water level threshold induces a pump not only to prevent reduction but also to keep a reserve for continuous operation. This condition is met through a conductive probe-type water level sensor because of its simplicity, low cost, and suitability for small tanks relative to ultrasonic alternatives. Mobility needs form the basis for the mobile housing concept, dictating the inclusion of a motorized chassis with DC motors and H-bridge drivers, as well as a solar-powered battery system supplying power for the motion. The GPS module is used primarily for real-time location tracking, as well as theft prevention and monitoring the movement of the housing system through the Blynk mobile application. This improves the system security and allows the caretaker to monitor the location of the housing unit remotely. Security and monitoring requirements, such as keypad-based access control, intrusion-triggered alarms, and real-time mobile application supervision, address the risks of theft and unauthorized access. These requirements shape the design by introducing a dedicated security subsystem with input devices, alert outputs, and a

persistent network connection for remote notifications. The keypad acts as the main authentication system, while the buzzer provides immediate alerts when incorrect password attempts occur. The decision to use Wi-Fi as the primary communication protocol is based on the need for continuous cloud data logging, real-time monitoring, and remote user interaction through the Blynk mobile application and InfluxDB cloud database. Lastly, data collection, transmission, and storage form the informational backbone of the system. Regular periodic time-stamped data logging enables long-term environmental monitoring, behavioral analysis, and future AI-based optimization. This justifies the use of the ESP32 microcontroller due to its sufficient processing capability, built-in Wi-Fi, low power consumption, and compatibility with multiple sensors and actuators, while also supporting the database structure needed for trend identification and system evaluation.

3.2 Analysis of Design Constraints

The proposed design is shaped and bounded by a set of realistic constraints that ensure its practicality, economic viability, and long-term sustainability. The foremost economic constraint of a 500 JD total budget is a decisive factor in every component selection and architectural decision. This financial limit prevents the use of expensive industrial sensors or complex single-board computers, directing the design toward a cost-optimized combination of reliable, commercially available components. For instance, the ESP32 microcontroller was selected instead of a Raspberry Pi because of its built-in Wi-Fi and Bluetooth capabilities, lower power consumption, and significantly lower cost. This allows more budget allocation for essential actuators such as motors, water pumps, and the solar power system. Each component was selected not only based on performance but also on its contribution to keeping the total system cost within the project's financial limit. This constraint also encourages design simplicity, where unnecessary complexity and redundant solutions are avoided. This is also one of the most important factors in the system design. Since the project is a mobile, solar-powered animal housing unit intended for outdoor operation, energy efficiency is not only beneficial but necessary for continuous functionality. This directly supports the integration of solar panels as the main power source. It also influences the selection of low-power sensors and efficient control logic to reduce unnecessary energy usage. The ESP32 was selected partly because of its low power consumption and deep

sleep capabilities, allowing the system to conserve energy between sensor polling intervals. Similarly, components such as the LDR-based light sensor and conductive water level sensor were selected because they consume less power compared to more complex alternatives. High-power actuators such as the water pump, fans, and movement motors are programmed to operate only when necessary, ensuring efficient use of battery and solar energy. Another important constraint is scalability and reproducibility. The system must be easy to reproduce using standard commercial components so that it can be implemented in larger farms or adapted for future expansion. This requires the use of common voltage levels such as 3.3V and 5V, standard communication protocols such as I2C, UART, and Wi-Fi, and simple mechanical structures. The use of readily available components also simplifies maintenance, replacement, and future upgrades. Environmental durability is also a major constraint because the system operates in outdoor farm conditions where it is exposed to rain, dust, sunlight, mud, and temperature changes. Therefore, the selected sensors and actuators must be reliable in outdoor environments. Features such as the automatic roof-closing mechanism during rainfall improve both system protection and animal safety. The mobility feature creates an additional engineering constraint. The floor-less housing structure must be moved daily using DC motors and H-bridge drivers while maintaining stability, safety, and power efficiency. This requires careful battery sizing, proper motor control, and mechanical support to ensure smooth movement without damaging the electronic system. Ultimately, these constraints do not limit the design but define its most practical and innovative features. They transform the project from a theoretical concept into a realistic prototype with strong potential for real-world agricultural deployment.

3.3 Discussion of Engineering Standards

The following engineering standards guide the implementation of the smart mobile animal housing system:

- **WiFi:** This standard enables wireless communication between the ESP32 microcontrollers and the Blynk IoT platform, InfluxDB cloud database, and Grafana virtualization server. It allows real-time remote monitoring of sensor data, such as temperature, water level, rain detection, and light intensity, manual and automatic actuator control, such as fans, water pump, and roof control, instant push notifications, such as low water or intrusion, cloud

data transmission with timestamped logging, and mobile/web dashboard visualization. Wi-Fi IEEE 802.11 ensures compatibility with standard access points and provides sufficient bandwidth for simultaneous sensor data streaming and bidirectional command transmission.

- **GPS:** This standard is used for real-time location tracking of the mobile animal housing system via the NEO-6M module. This standard will allow tracking of the system geographical coordinates (latitude and longitude), so that the owner knows the exact location of the system wherever it goes. It has a 2.5-meter accuracy which is sufficient for this project.
- **I2C Communication Protocol:** This standard is used for communication between the ESP32 and the LCD display. This protocol with 2 wires (SDA and SCL) minimizes pin usage while providing reliable data transmission for displaying the access code.
- **UART Serial Communication:** This standard is used for communication with the GPS module. It allows receiving NMEA sentences that contain longitude and latitude data, communication between peripherals and ESP32 at 9600 baud rate, and reliable asynchronous data transmission without clock synchronization.

3.4 Different Design Approaches

Building on some shortcomings, the proposed system implements a re-designed architecture aimed at enhancing energy efficiency, robustness, and autonomous decision-making. The integration of solar panels also allows for renewable, off-grid power and long-term outdoor operation. The sensors employed in the proposed system are low cost and high accuracy. The LM35 temperature sensor was adopted due to its stable linear output, low cost, and reliable performance in outdoor environments. The LM35 needs less wiring and lower processing overhead, and thus is a better fit for use in an energy-efficient, lightweight system. The Raindrop Detector Sensor (YL-83) uses a conductive rain board that changes resistance when droplets fall on it, making it possible for extremely fast and reliable detection of rain or snow. Compared to commercial weather stations or capacitive rain sensors, the YL-83 is significantly cheaper and simpler to integrate while still providing adequate accuracy for farm applications. The Light Sensor Module (LDR) detects ambient light by changing resistance depending on illumination levels. The built-in potentiometer allows a threshold to be set for digital switching, enabling fine-tuning under different lighting

conditions. Unlike digital light sensors such as the BH1750, the LDR module is far more cost-effective and consumes less power, making it ideal for basic day–night detection. The probe-type water level sensor provides multiple level readings based on the conductive traces that detect water presence at different heights. It is low cost, lightweight, and consumes very little power. In contrast to ultrasonic or float-switch systems, this probe sensor offers simpler installation and multi-level output at a much lower cost, which is suitable for a small water tank. The NEO-6M GPS module offers high positioning accuracy (typically within 2.5 meters) and a high-sensitivity receiver capable of maintaining satellite lock even in cloudy or partially obstructed environments. Furthermore, it is known for low power consumption which matches the system’s energy-efficient design. Compared to more advanced GPS modules, the NEO-6M provides an excellent balance of accuracy, power efficiency, and cost, which makes it the optimal option for IoT applications where location tracking is necessary. Due to its ability to sense, control, and communicate within a low-power platform, the ESP32 microcontroller was selected as the central processing unit. Compared to Arduino Uno and PIC microcontrollers, the ESP32 offers higher processing capabilities, enabling concurrent execution of sensor data acquisition, automated control algorithms, and communication tasks. Unlike the Raspberry Pi, the ESP32 consumes less power, making it more suitable for long-term outdoor operation. The built-in WiFi and Bluetooth Low Energy (BLE) eliminate the need for external communication hardware, reducing the system’s complexity, cost, and power consumption. Overall, the ESP32 provides the optimal balance between performance, power efficiency, cost, and scalability.

3.5 Design Development

The system is divided into two ESP32-S3-N16R8 to separate tasks and reduce processing load. The ESP32-S3-N16R8 is a high-performance, AI-focused module featuring an Espressif ESP32-S3 dual-core microcontroller, 16MB of Flash, and 8MB of Octal SPI PSRAM which is much larger than standard ESP32 boards. It has built-in WiFi and Bluetooth.

The ESP32-S3 N16R8 supports sleep mode, deep sleep and efficient battery usage making it a good choice for outdoor systems. The first ESP32-S3-N16R8 handles environmental monitoring, including temperature, rain detection, and light intensity as shown in Figure 3-1.

First ESP32 Architecture: Environmental Monitoring, Actuator Control, and Cloud Integration

Based on the uploaded ESP32 code: temperature, light, motion, rain, GPS, fans, LED, buzzer, H-bridge roof control, Blynk, InfluxDB, and Grafana

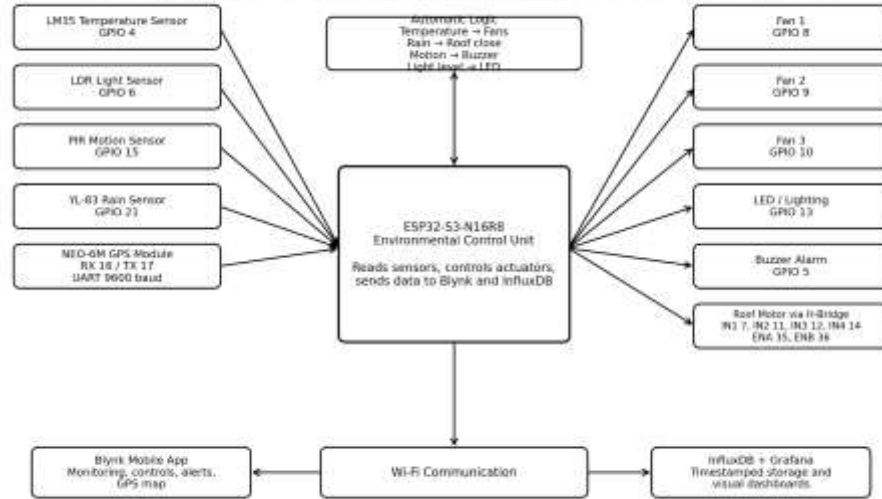


Figure 3-1: First ESP32 Architecture.

The second ESP32-S3-N16R8 deployed handles intrusion detection and water management and mobility control as shown in Figure 3-2. All sensors on both microcontrollers collect data from the environment and send those readings to the ESP32 boards. Actuators then respond accordingly. WiFi is used as the main communication method between the ESP32 boards, Blynk and InfluxDB.

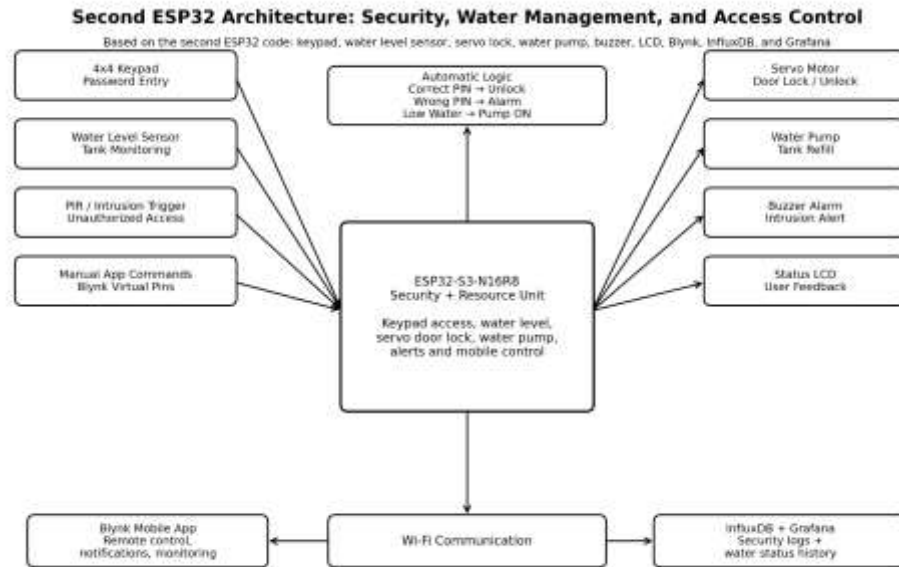


Figure 3-2: Second ESP32 Architecture.

Blynk is a low-code IoT (Internet of Things) platform designed to connect hardware to the cloud, allowing the user to build mobile apps and web dashboards to control and monitor his/her devices remotely. It allows real-time monitoring and manual remote control. Unlike simple dashboards, Blynk allows direct control of actuators. Blynk also sends real-time alerts. It supports GPS map widgets, allowing users to track the exact location of the mobile housing unit. InfluxDB is a time-series cloud database designed for storing data that changes over time. InfluxDB uses API tokens to prove that the ESP32 is allowed to write data onto the InfluxDB account. The API token provides authentication, authorization and security. InfluxDB is connected to Grafana which is a data visualization platform. Grafana visualizes real-time and historical data using dashboards, making the system behavior easier to monitor and analyze. The solar power system supports outdoor operation. The solar panels charge batteries that turn everything on. The architecture combines automation, mobility, renewable energy, security and data logging into one integrated system. Figure 3-3 shows the overall architecture of the system.

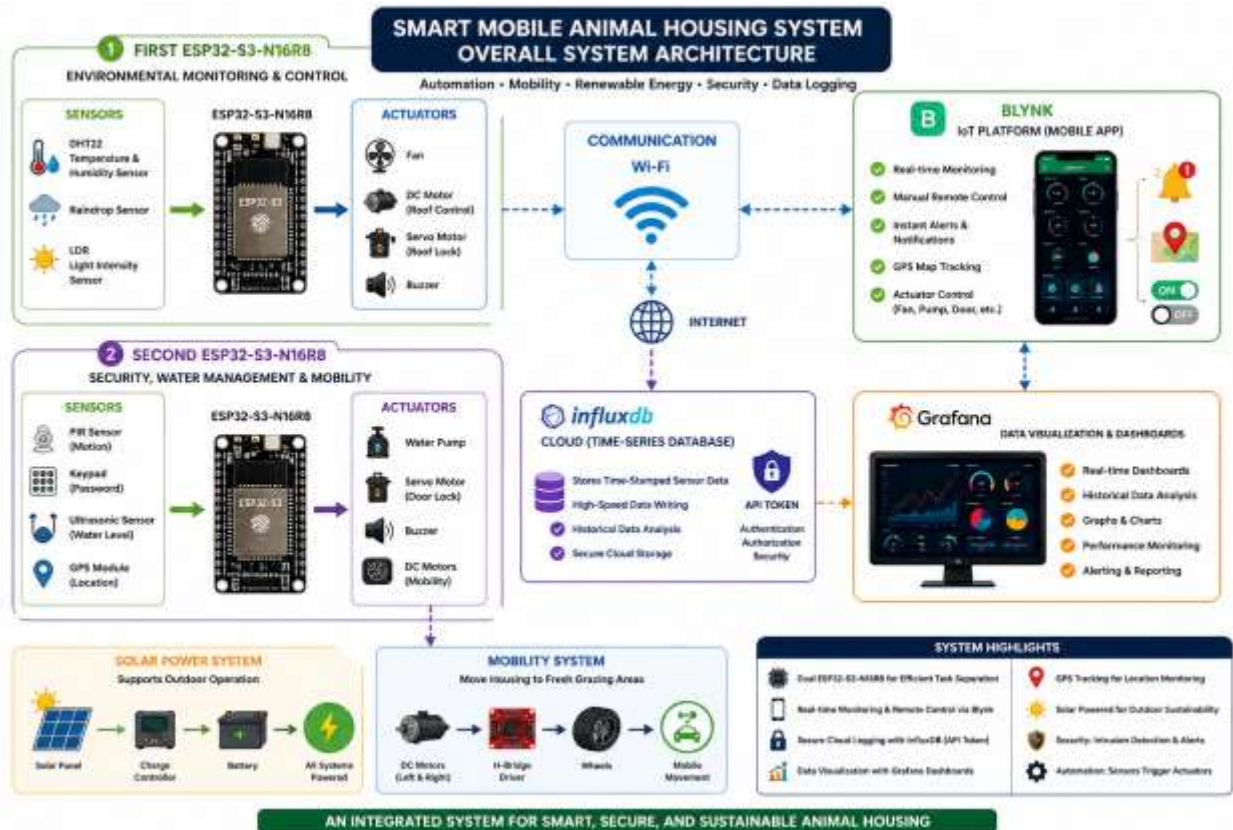


Figure 3-3: Overall Architecture.

3.5.1 Environmental Control System

This subsystem's main purpose is to maintain safe and comfortable living conditions for the animals. It automatically regulates temperature, lighting, airflow, and weather. It prevents heat stress, darkness, and exposure to rain.

This subsystem incorporates several sensors to enable continuous monitoring of environmental conditions. The temperature sensor continuously monitors the thermal conditions, allowing the system to activate the ventilation mechanisms when a predefined threshold is exceeded. The light sensor assesses the illumination levels to determine whether to turn on the lights in the shelter or keep them off. The rain sensor is incorporated to detect rainfall, allowing control of the roof mechanism to protect the welfare of the animals. Figure 3-4 shows how the system is deployed.



Figure 3-4: Environmental Control Subsystem.

3.5.2 Resource Management System

This subsystem is designed to ensure continuous availability of resources and operational control within the mobile animal shelter. The water level sensor is integrated to monitor the tank's capacity, enabling the system to detect when water level is under a certain threshold to activate the water pump. The GPS module is deployed to track the geographical position of the shelter, allowing the system to update its location and support scheduled mobility routines. Figure 3-5 shows the deployment of this system.



Figure 3-5: Resource Management Subsystem.

3.5.3 Security and Intrusion Detection System

This subsystem is implemented to protect the animal shelter from unauthorized access, intrusion, and theft. The keypad serves as the primary authentication interface, allowing only authorized users with the correct access code to enter the shelter. When the correct passcode is entered, the servo-controlled door unlocks to grant access. In addition, the PIR motion sensor detects movement around the shelter and activates a buzzer alarm when intrusion is detected. Figure 3-6 illustrates the deployment of the subsystem.



Figure 3-6: Security and Intrusion Detection Subsystem.

3.5.4 Communication and Cloud Integration

Wi-Fi is the primary communication protocol for data transmission between the mobile animal housing system and the cloud platform. The ESP32-S3-N16R8 microcontroller establishes a connection to a local access point and communicates with two primary cloud services: Blynk and InfluxDB.

Blynk IoT Platform: Blynk serves as the primary user interface and real-time control layer. The ESP32-S3 authenticates with Blynk cloud using an authentication token and periodically pushes sensor readings (temperature, water level status, rain detection, motion detection) to virtual pins. These readings are immediately displayed on the Blynk dashboard, which the caretaker can access from any device with internet connectivity. Blynk also handles bidirectional communication. When a user interacts with the dashboard, commands are sent through Blynk's real-time event system to the ESP32-S3, which executes the corresponding action. In addition, Blynk's Event Widget is configured to send push notifications for critical alerts, such as intrusion detection and low water level. The GPS location data is sent to Blynk's Map Widget, displaying the housing's real-time position and movement history. It also displays the random code generated by the ESP32-S3 so that the user enters the code and the door unlocks if it were correct.

InfluxDB Time-Series Database while Blynk provides real-time monitoring and control, long-term data storage and advanced analytics are assigned to InfluxDB, a purpose-built time series database. Sensor readings were transmitted through WiFi using HTTP POST requests and formatted using the InfluxDB line protocol. Each data entry is tagged with metadata and timestamped using the ESP32-S3's internal real-time clock, which is synchronized via NTP over Wi-Fi. InfluxDB's API token system provides authentication and authorization to make sure that only the authorized ESP32-S3 can write data to the database. This separation of concerns helps in avoiding data congestion on the Blynk platform.

Grafana Visualization: Grafana is linked to InfluxDB as a data source, which provides customizable data dashboards for visualizing historical and real-time data. Unlike Blynk's simple widget, Grafana enables the caretaker to make time-series graphs displaying temperature trends over multiple days or weeks, correlate rain events to changes in temperature, and monitor water consumption patterns. These dashboards fulfill the system's need for data analysis by enabling identification of environmental trends and energy consumption optimization. To conserve power and reduce Wi-Fi duty cycles, the ESP32-S3 buffers sensor readings locally during certain time periods and transmits batches of data at predefined times. The urgency for quick high-priority transmission is immediately triggered by critical events through Blynk's Event system, allowing alerts to reach the caretaker in short notice.

3.5.6 Power Management

In this system, the main power source used is a solar panel. The solar panel generates electrical energy from sunlight. The solar panel is connected to a solar charge controller that regulates the charging process. The solar charge controller performs several important functions that include Battery Protection prevents deep discharge, battery failure and short battery lifespan.

Stable Output Power: The USB output gives stable 5V output, ideal for powering ESP32

The battery connected to the solar charge controller stores energy. The ESP32 boards are then connected to the 5V USB output. Figure 3-7 illustrates how all components mentioned are connected.

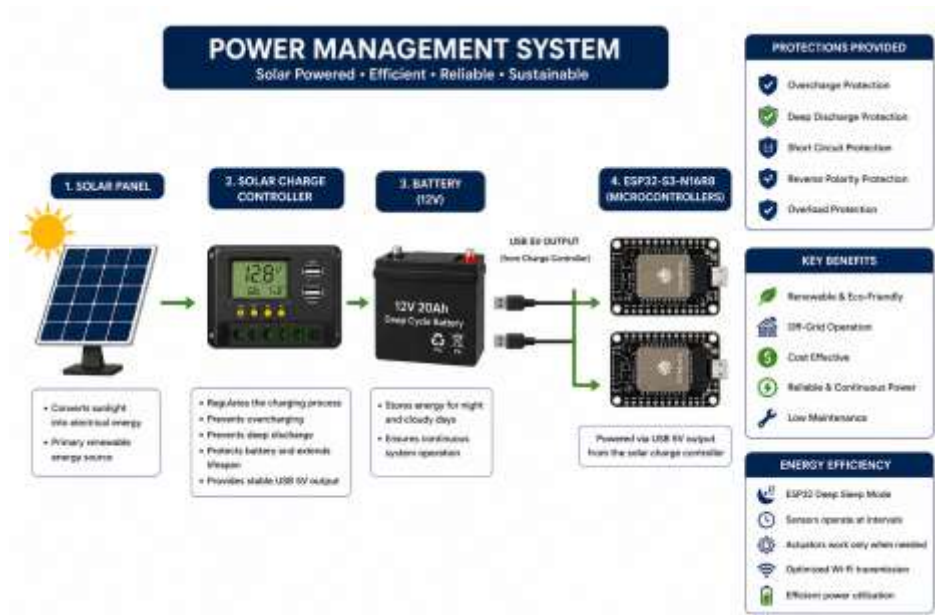


Figure 3-7: Power Management.

3.5.7 Cost Analysis

The total bill of materials (BOM) should not exceed the 500 JOD budgetary constraint while maintaining functionality and reliability. Table 3-1 provides details about the total amount of money that was spent on this project.

Table 3-1: System Components and Prices.

Components	Type	Quantity	Unit Price (JOD)	Total (JOD)
Temperature Sensor	LM35	1	3.25	3.25
Microcontroller	ESP32-S3-N16R8	2	10.90	21.80
GPS Module	NEO-6M	1	13.00	13.00
H-Bridge Driver	L298N	1	2.70	2.70
Keypad	4x3 Matrix Keypad	1	3.40	3.40
Servo Motor	SG90	1	2.55	2.55
Water Pump	DC 5V Submersible Pump	1	8.00	8.00
Water Level Sensor	XKC-Y25-NPN	1	12.00	12.00
Rain Sensor	YL-83	1	1.90	1.90
Light Sensor	LDR	1	1.00	1.00
Buzzer	5V Magnetic Buzzer	1	0.25	0.25
Breadboard	400-Point Solderless Breadboard	4	1.75	7.00
Jumper Wires	Male-Male/ Female-Female	4	0.90	3.60
Fans	Mini DC Fans	3	5.00	15.00
Solar Charge Controller	30A Solar Charge Controller	1	12.00	12.00
Battery	12V 7Ah Lead Acid Battery	1	16.50	16.50
Solar Panel	10W Monocrystalline Solar Panel	1	24.00	24.00
LED Kit	3mm/5mm LED Kit	1	3.50	3.50
LCD Display	16x4 LCD Module	1	6.00	6.00

The total estimated cost is 164.45 JOD

4 Results

4.1 Prototype Setup

The prototype was implemented using two ESP32-S3-N16R8 microcontrollers to automate the entire smart mobile animal housing system. The two microcontrollers were used to distribute system tasks and reduce processing overload and improve system performance. The prototype integrates multiple sensors and actuators to provide automated environmental monitoring, security, water management, mobility control, and real-time remote monitoring

4.1.1 Hardware Setup

The first ESP32-S3-N16R8 microcontroller is primarily responsible for environmental monitoring and control, it is the main processing unit. The connected sensors and actuators include:

- LM35 temperature sensor for temperature monitoring
- LDR sensor for light intensity detection
- YL-83 rain sensor for rain detection
- GPS module (NEO-6M) for real-time location tracking
- Fans for ventilation and cooling
- Roof movement mechanism using an H-bridge and DC motor
- PIR sensor module for motion detection
- Buzzer for alert generation

The second ESP32-S3-N16R8 microcontroller is responsible for security, water management, and mobility control. The connected sensors and actuators include:

- Water level sensor for monitoring the tank level
- Water pump for automatic tank refilling
- Keypad (4x3 matrix) for password-based access control
- Servo motor for door locking and unlocking
- Buzzer for intrusion alerts and incorrect password indication
- DC motors for movement of the housing system
- H-bridge motor driver for controlling motor direction and speed

Table 4-1 and Table 4-2 present the GPIO pin assignments for all sensors and actuators connected to each ESP32 microcontroller.

Table 4-1: First ESP32 Components.

Components	GPIO Pins	Type	Description
LM35 Temperature Sensor	GPIO 4	Analog	Measures temperature
LDR	GPIO 6	Analog	Detects light intensity
PIR Sensor	GPIO 15	Digital	Detects motion
Rain Sensor (YL-83)	GPIO 21	Digital	Detects rain presence
GPS RX	GPIO 16	UART	Receives GPS data
GPS TX	GPIO 17	UART	Sends GPS data
Servo Motor 1	GPIO 18	PWM	Roof movement control
Servo Motor 2	GPIO 19	PWM	Roof movement control
Buzzer	GPIO 5	Digital	Alert output
Fan 1	GPIO 8	Digital	Fan control
Fan 2	GPIO 9	Digital	Fan control
Fan 3	GPIO 10	Digital	Fan control
LED	GPIO 13	Digital	Status indication

Table 4-2: Second ESP32 Components.

Components	GPIO Pins	Type	Description
Water Level Sensor	GPIO 10	Digital	Detects water level
Water Pump Relay	GPIO 21	Digital	Controls water pump
Servo Motor	GPIO 13	PWM	Controls water pump
Keypad Row 1	GPIO 11	Digital	Keypad input
Keypad Row 2	GPIO 12	Digital	Keypad input
Keypad Row 3	GPIO 14	Digital	Keypad input
Keypad Row 4	GPIO 15	Digital	Keypad input
Keypad Column 1	GPIO 18	Digital	Keypad input
Keypad Column 2	GPIO 5 19	Digital	Keypad input
Keypad Column 3	GPIO 20	Digital	Keypad input
LCD SDA	GPIO 8	I2C	2C Data Line
LCD SCL	GPIO 9	I2C	2C CLK Line
H-Bridge IN 1	GPIO 4	Digital	Motor direction control
H-Bridge IN 2	GPIO 8	Digital	Motor direction control
H-Bridge IN 3	GPIO 7	Digital	Motor direction control
H-Bridge IN 4	GPIO 6	Digital	Motor direction control
H-Bridge EN A	GPIO 16	PWM	Motor speed control
H-Bridge EN B	GPIO 17	PWM	Motor speed control

The system is powered using a solar energy setup consisting of a solar panel, a charge controller, and rechargeable batteries. The ESP32 microcontrollers are powered via the USB output of the charge controller, which provides a stable 5V supply. High power components such as DC motors require higher current and are therefore powered directly from the battery to ensure reliable operation.

4.1.2 Software Setup

Both microcontrollers were programmed using Visual Studio Code with the PlatformIO extension. PlatformIO is a cross-platform, cross-architecture, professional ecosystem for embedded software development, designed to streamline IoT and microcontroller projects. It works primarily as a VSCode extension, providing advanced features like intelligent code completion, debugging, unit testing, and support for over 1,000+ boards. Many libraries were installed for different sensors that include:

- Blynk Library
- WiFi Library
- TinyGPS++ Library
- Keypad Library
- ESP32Servo Library
- InfluxDB Client Library

Blynk was used as the primary platform for real-time monitoring and manual control of the system. Because two ESP32 microcontrollers were used, two distinct Blynk devices were generated, each corresponding to one subsystem. An authentication token (Auth Token) was assigned to each ESP32 to provide secure communication with the Blynk cloud. For each sensor and actuator, virtual pins were set up for sending data and monitoring via the dashboard. The Blynk web dashboard was designed using several widgets, including labels for showing sensor data and buttons for manual actuator control. There were also events configured inside Blynk to generate alerts in real time, as well as notifications for specific conditions like the presence of rain, low water levels, and unauthorized access attempts. In this approach, a scalable and robust solution enables efficient real-time monitoring and control while ensuring separation of system subsystems. Event notifications were set to send alerts for showing sensor data and buttons for manual actuator

control. There were also events configured inside Blynk to generate alerts in real time, as well as notifications for specific conditions like the presence of rain, low water levels, and unauthorized access attempts. In this approach, a scalable and robust solution enables efficient real-time monitoring and control while ensuring separation of system subsystems.

Event notifications were set to send alerts for:

- Intrusion detection
- Low water level
- High temperature

Sensor data was stored in InfluxDB as the cloud-based time-series database. Sensor readings were transmitted through WiFi using HTTP POST requests and formatted with the InfluxDB line protocol. Each ESP32 writes data to its dedicated database bucket using secure API token authentication. Grafana was connected to InfluxDB to visualize both real-time and historical sensor data. The Grafana dashboards display environmental trends, water usage statistics, motion detection logs, and system activity records using time-series graphs and visual indicators.

Figure 4-1 illustrates the communication workflow between Blynk, InfluxDB, and Grafana.

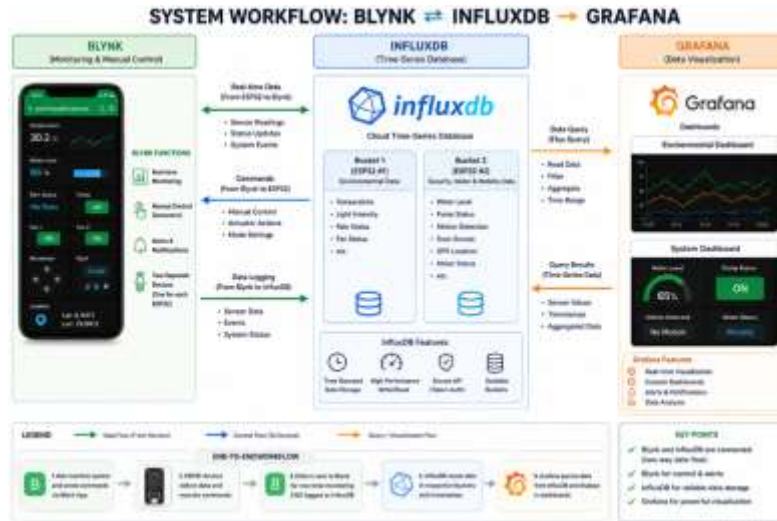


Figure 4-1: System Workflow of Blynk, InfluxDB, and Grafana.

4.2 Testing

Several experiments were performed to validate environmental monitoring, actuator control, cloud communication, and security.

• Temperature Monitoring Test

To test the LM35 temperature sensor, the sensor was exposed to different temperature conditions to evaluate the environmental monitoring functionality of the system. The prototype deploys three DC fans for automatic ventilation and cooling. Depending on the measured temperature, the ESP32 automatically activates or deactivates the fans once the predefined temperature threshold is reached. The temperature readings were successfully displayed on the Blynk dashboard in real time.

• Rain Detection Test

To test the YL-83 rain detection sensor, water droplets were applied to the sensor surface to simulate rainfall conditions and verify the roof control mechanism. The roof mechanism was implemented using two servo motors. Once rain was detected, the ESP32 automatically triggered the roof to close the roof automatically once rain was detected.

• Motion Detection Test

The PIR motion sensor was tested by introducing movement near the housing system. The system successfully detected motion and generated intrusion alerts through the buzzer and Blynk notification system.

- **Water Level Monitoring Test**

The water level sensor was tested by varying the water level inside the tank. When the water level dropped below the threshold, the water pump was automatically activated to refill the tank.

- **Keypad Security Test**

The keypad access system was tested using both correct and incorrect passwords. Correct passwords unlocked the servo-controlled door, while incorrect entries triggered the buzzer alarm and intrusion notifications.

- **GPS Tracking Test**

The NEO-6M GPS module was tested outdoors to verify real-time location tracking. GPS coordinates were successfully displayed on the Blynk map widget.

- **Cloud Communication Test**

WiFi communication with Blynk and InfluxDB was tested to verify successful cloud connectivity, remote monitoring, and data transmission. Sensor readings were successfully transmitted and stored in the cloud database.

4.3 Discussion of Results

The prototype realized the primary goals of the project by integrating automation, environmental monitoring, security, mobility, and renewable energy in this single smart housing system. To this end, an environmental monitoring subsystem managed to control temperature, lighting, and rain protection with automated sensor/actuator interaction. The fan activation mechanism decreased extreme temperature, while the roof control system allowed protection against rainfall. The security subsystem integrated keypad-based access control and intrusion detection. In case of unauthorized access attempts, buzzer alerts and mobile notification were also activated through the Blynk platform. Water level monitoring and automated processes were accomplished successfully by the resource management subsystem, which controlled the water pump operation. Also, the mobile dashboard allowed real-time housing location tracking thanks to GPS tracking. The Blynk integration made real-time monitoring efficient and remote manual control of actuators

within the system a breeze. Historical sensor data was stored and visualized successfully in InfluxDB and Grafana for long-term system analysis and trend monitoring. The Blynk dashboards allow the user to control the system and monitor the data from anywhere in the world as long as the user has internet connectivity. The user can turn on/off fans, can see the temperature, can turn on/off the water pump, can see if there is rain or not, see if any motion is detected, can open/close roof, see the light intensity, and receive the PIN for the door access and see if the password entered is correct or not, and can see the water level.

Figure 4-2 shows the dashboard.

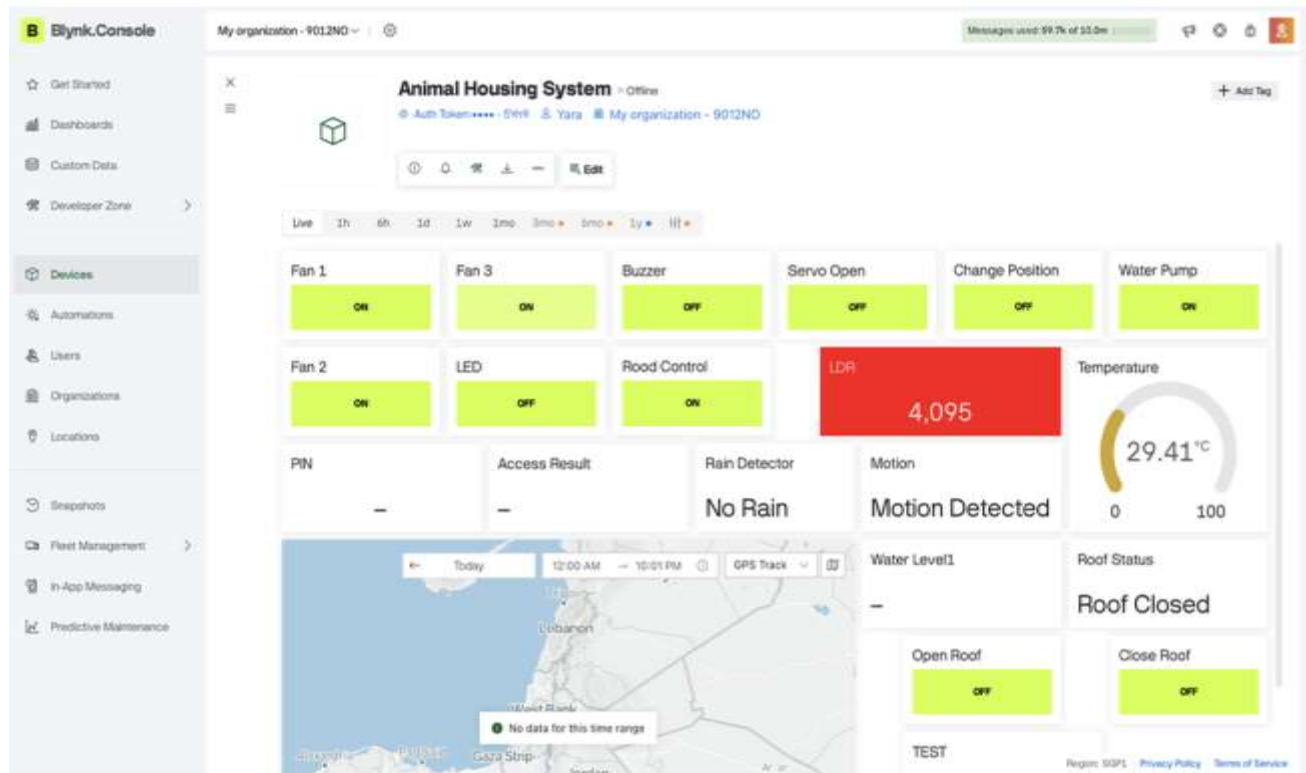


Figure 4-2: Blynk Dashboard.

The Grafana dashboard virtualizes the data collected from each sensor and displays a report of each sensor and actuator. Figure 4-3 displays the Grafana dashboard.

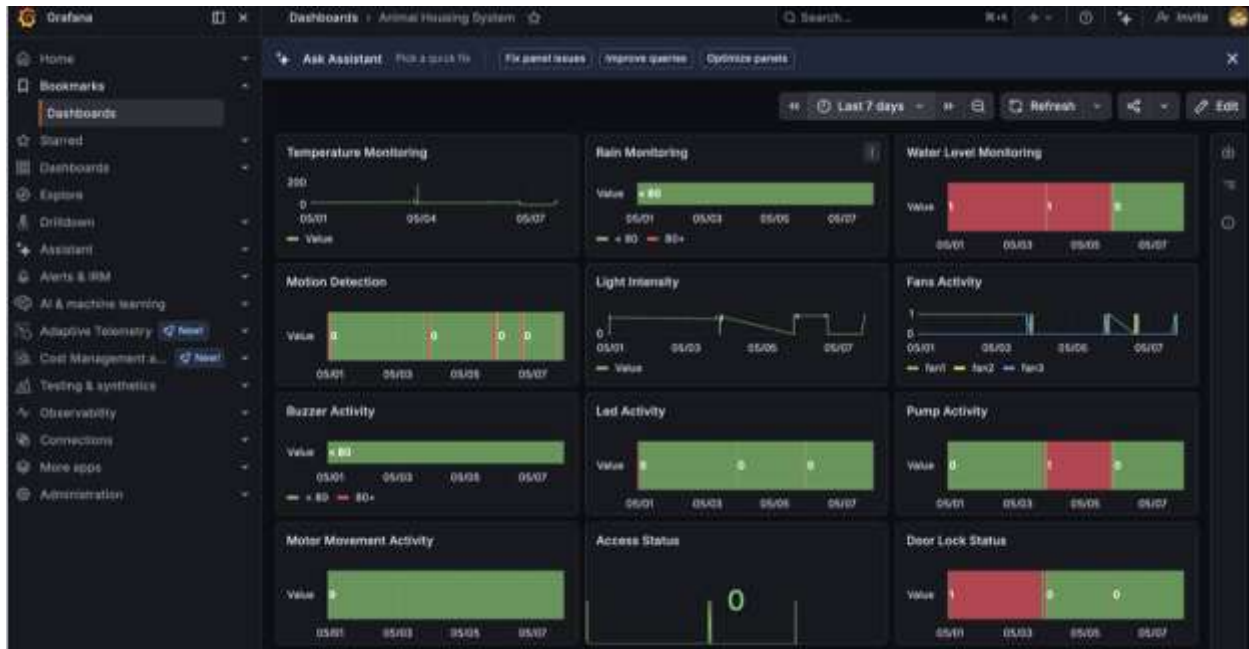


Figure 4-3: Grafana Dashboard.

4.4 Validation of Design Requirements within the Realistic Constraints

Table 4-3 validates the design requirements.

Table 4-3: Validation of Design Requirements.

Requirements	Validation Method	Result
Data Collection	Tested LM35, YL-83 rain sensor, water level sensor, PIR sensor, and GPS module	All sensors collected data successfully. LM35 achieved approximately $\pm 0.5^{\circ}\text{C}$ accuracy. Rain sensor reliably detected water presence, and the water level sensor detected two distinct levels accurately.
Data Transmission	Tested transmitting data from both ESP32s to Blynk and InfluxDB	Sensor data was transmitted successfully over IEEE 802.11 WiFi with average latency below 500 ms. Data packets were correctly

		time-stamped.
Automated Control and Response	Tested fans, water pump, smart door, roof, and DC motors responses	Fans activated above 30°C. Water pump activated below 25% water level. Smart door unlocked only with the correct 4-digit password. The roof closed automatically during rain detection. DC motors moved the housing structure successfully.
Budget Constraints	Summed up the total amount of money spent	Total prototype cost was approximately 500 JOD
Power consumption	Tested if the system worked with low power	The system operated continuously with low power consumption.

5 Conclusions and Future Work

The aim of this project was to develop the IoT-based smart mobile animal housing system that enhances animal welfare, reduces manual labour, and boosts operational efficiencies by automation, real-time analysis with renewable power. Our key findings showed that our system automatizes environmental control (temperature controlled fans, rain responsive roof closure, light-activated LED lighting), resource management (automatic refill of water tank at <25% water level) and security (keypad based access with buzzer notifications for failed attempts). Blynk and data in real-time monitoring. Grafana dashboard visualization offering complete remote supervision. The dual-ESP32 architecture distributed processing works independently without overload. Solar power enabled continuous outdoor operation. This innovative work provides major contributions into the smart agriculture field with a mobile intelligent floorless animal housing system that directly addresses overgrazing and waste accumulation—problems not attended to by fixed smart pet houses in previous literature. The integration of GPS tracking with keypad security for more location-aware, theft-resistant platforms. The use of InfluxDB time series database with Grafana visualization for long run trend analysis, beyond typical threshold-based automation. The results were made subject to a variety of constraints: the 500 JD budget restricted the use of higher precision sensors (such as ultrasonic water level sensors) and industrial-grade GPS units. Prototype mobility was evaluated on level grass lawns but not on uneven earth or mud ground. Wi-Fi dependency relies on farm connectivity to prevent this failure, which may not be practical in very distant areas. Future improvements include:

1. Cellular connectivity (4G/LTE) using SIM800L modules for remote farms without internet infrastructure.
2. AI-based behavioral analysis to detect abnormal animal movement patterns
3. Automated feeding system with a servo-controlled hopper
4. Multi-unit coordination for large-scale farms.

The application can be expanded in other areas, e.g., poultry transport crates, emergency animal shelters during floods or fires, or temporary veterinary clinics in remote locations. Research directions to guide future work include optimization of solar-to-mobility energy

efficiency and development of predictive grazing algorithms based on historical GPS and pasture growth data.

References

- [1] D. Fraser, R. M. Kharb, C. McCrindle, J. Mench, M. Paranhos da Costa, K. Promchan, A. Sundrum, P. Thornber, P. Whittington, and W. Song, "Capacity building to implement good animal welfare practices," Food and Agriculture Organization of the United Nations, Tech. Rep., 2009.
- [2] M. Bovo, S. Benni, A. Barbaresi, E. Santolini, M. Agrusti, D. Torreggiani, and P. Tassinari, "A smart monitoring system for a future smarter dairy farming," in 2020 IEEE International Workshop on Metrology for Agriculture and Forestry (MetroAgriFor), 2020, pp. 165–169.
- [3] P. C. Goo, K. G. Tay, C. C. Chew, and A. Huong, "Smart pet house," in 2024 International Conference on Future Technologies for Smart Society (ICFTSS), 2024, pp. 37–42.
- [4] Y.-C. Tung, Y.-H. Lien, L.-W. Liu, M. D. Honesty B, and P. K. Devi, "Iot-smart monitoring of pet housing," in 2024 International Conference on Wireless Communications Signal Processing and Networking (WiSPNET), 2024, pp. 1–6.
- [5] A. Mandy, H. Qazweeni, M. Nouredine, T. Al-Radhwan, and M. El-Abd, "Smart pet house," in 2016 International Conference on Industrial Informatics and Computer Systems(CIICS), 2016, pp. 1–5.
- [6] B. Zionetha Mailoa, K. Wiemas Naufal Ghiffari, D. Mahdi Asriny, Y. Ernawati, and N. Legowo, "Design of internet of things for smart residence," in 2022 International Conference on ICT for Smart Society (ICISS), 2022, pp. 01–06.
- [7] W. Y. Leong, "Smart poultry farming 4.0: Leveraging iot for climate control and productivity," in 2025 21st IEEE International Colloquium on Signal Processing Its Applications (CSPA), 2025, pp. 172–175.

Appendices

Appendix A: Offering Template



Princess Sumaya University for Technology King Abdullah II School of Engineering Senior Design Projects

Design and Implementation of an IoT-Based Smart Mobile Animal Housing System

1	Advisor Name	Dr. Rami Abdullah Wahsheh
	Co-Advisor Name	
2	Year / Semester	First Semester 2025-2026
3	Title of Senior Design Project	Design and Implementation of an IoT-Based Smart Mobile Animal Housing System
4	Design to be Achieved	This project involves the design and implementation of an IoT-based smart mobile animal housing system that features a floorless structure and is equipped with solar-powered motorized wheels, enabling autonomous relocation to new sites each day with customizable settings. This design ensures that the animals have continuous access to fresh grass and a clean environment, while also preventing manure accumulation and promoting natural physical activity. The system is designed to automatically manage smart lighting, ventilation fans, and environmental conditions within the housing. It can detect the time of day and motion, automatically turning on the lights when movement or human presence is detected. Additionally, the system features a smart entry porch that detects the presence of individuals at the door and prompts a secure password verification for entry. If the correct password is entered, the door unlocks automatically; otherwise, an intrusion alarm is triggered. The system also includes an advanced intrusion detection mechanism, particularly monitoring the roof area, which activates a burglar alert in the event of unauthorized access. An optional automated water management function allows the control of a water pump to refill the tank, with an ultrasonic sensor monitoring the water level in real time. Furthermore, the system integrates environmental sensors, including a temperature sensor, solar intensity tracker, and rain sensor for weather monitoring and adaptive control. All collected data is time-stamped and stored by day and hour, allowing the system to analyze user behavior and operational patterns. This data serves as the foundation for future AI-based recommendation features aimed at optimizing energy efficiency, improving security, and simplifying daily management tasks. The system's overall functionality can be monitored and controlled through a centralized, user-friendly mobile or web application. Based on user analytics, the app may send intelligent notifications, such as: "The lights are usually off at this time. Would you like to turn them off now?"
5	Engineering Standard	WiFi, GPS, and Bluetooth
6	Design Requirements	The design shall achieve the following requirements: 1-The operating voltage for all sensors is between 3 to 5 volts. 2-The sensor's data is sent every 10 minutes (adjustable). 3-The data is stored in time frames and days of the week for the system to learn the environment behavior and user habits. 4-All components should operate within the industrial temperature range 40°C to +85 °C. 5-An alert is sent when the water tank level is below 25% of its capacity. 6-An alert is sent when an intruder is inside the coop. 7-The fan is turned on when the temperature is above 300oC and automatically turns off when the temperature is below 290oC. 8-Based on the width of the coop, the coop moves a distance greater than its width by 1 meter (adjustable).

		9-Based on the length of the coop, the coop moves a distance greater than its length by 1 meter (adjustable)	
7	Realistic Constrains	Economic	Up to 500 JDs
		Environmental	Minimum power consumption
		Manufacturability and Sustainability	Should be scalable and built using available commercial components
		Other	
8	Deliverables	Documentation	Yes
		Portfolio	
		Video	
		Other	Google Maps
9	Background of Students	Communication Engineering, Electrical Engineering, and Computer Engineering Students.	
10	Number of Students	3 students	

Advisor Signature

Co-Advisor Signature

Department Head Signature

Appendix B: Code

ESP32 Code 1:

```
#define BLYNK_PRINT Serial
#define BLYNK_TEMPLATE_ID "TMPL6YKdWBVUb"
#define BLYNK_TEMPLATE_NAME "Animal Housing System"
#define BLYNK_AUTH_TOKEN "mSd_6qYXKe0pSoOd3PN4OrxVMzqj5Yn9"

#define INFLUXDB_URL "https://us-east-1-1.aws.cloud2.influxdata.com"
#define INFLUXDB_ORG "animal housing"
#define INFLUXDB_BUCKET "animal_housing"
#define INFLUXDB_TOKEN "52
hrfzPRKW_4T5_wN4WXoA4N6lxGLfGzmoHiB09c1Gud4HEhe0r3ZXNyqb9joYp8dwCjTwFWr3
W56HZtezITAw=="

#include <WiFiClientSecure.h>
#include <Arduino.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <ESP32Servo.h>
#include <TinyGPS++.h>
#include <InfluxDbClient.h>
#include <InfluxDbCloud.h>

#define TZ_INFO "EET-2EEST,M3.5.5/0,M10.5.5/1"

InfluxDBClient influxClient(
  INFLUXDB_URL,
  INFLUXDB_ORG,
  INFLUXDB_BUCKET,
  INFLUXDB_TOKEN,
  InfluxDbCloud2CACert
);

Point animalData("animal_house");

char ssid[] = "HUAWEI-2.4G-jNmV";
char pass[] = "96JwMPDs";

#define GPS_RX 16
#define GPS_TX 17
#define GPS_BAUD 9600
```

```

#define TEMP_PIN      4
#define LDR_PIN       6
#define PIR_PIN       15

#define SERVO_PIN     18
#define SERVO_PIN2    19

#define RAIN_SENSOR_PIN 21

#define FAN1          8
#define FAN2          9
#define FAN3          10
#define BUZZER        5
#define LED1_PIN      13

Servo roofServo;
Servo roofServo2;
HardwareSerial gpsSerial(2);
TinyGPSPlus gps;

#define SERVO_OPEN_ANGLE  90
#define SERVO_CLOSED_ANGLE 0
#define SERVO2_OPEN_ANGLE 180
#define SERVO2_CLOSED_ANGLE 90

#define V_TEMP          V10
#define V_LIGHT         V11
#define V_MOTION        V13
#define VPIN_RAIN       V22
#define V_GPS_MAP       V8

#define V_FAN1_CONTROL  V1
#define V_FAN2_CONTROL  V2
#define V_FAN3_CONTROL  V3
#define V_LED_CONTROL   V7
#define V_BUZZER_CONTROL V4
#define V_ROOF_OPEN     V26
#define V_ROOF_CLOSE    V27
#define V_ROOF_STATUS   V25
#define TEST            V28
#define V_GPS_SATS      V30
#define V_GPS_COORD     V31

```

```

#define V_GPS_DEBUG    V32
#define V_GPS_STATUS   V33

float temperatureC = 0;
int  lightValue    = 0;
bool  motion       = false;
bool  rainDetected = false;

bool manualFan1 = false;
bool manualFan2 = false;
bool manualFan3 = false;
bool manualLED   = false;
bool manualBuzzer = false;

bool tempAlertSent = false;
bool lastRainState = false;
bool roofsOpen = false;

void OpenRoof() {
  if (roofsOpen) {
    Serial.println("Roof already open, ignoring command");
    return;
  }

  Serial.println("=== OPENING ROOF ===");
  roofServo.write(SERVO_OPEN_ANGLE);
  roofServo2.write(SERVO2_OPEN_ANGLE);
  delay(500);
  roofsOpen = true;

  Blynk.virtualWrite(V_ROOF_STATUS, "Roof Open");
  Serial.println("Roof opened and staying open");
}

void CloseRoof() {
  if (!roofsOpen) {
    Serial.println("Roof already closed, ignoring command");
    return;
  }

  Serial.println("=== CLOSING ROOF ===");
  roofServo.write(SERVO_CLOSED_ANGLE);
  roofServo2.write(SERVO2_CLOSED_ANGLE);
  delay(500);
}

```

```

    roofsOpen = false;

    Blynk.virtualWrite(V_ROOF_STATUS, "Roof Closed");
    Serial.println("Roof closed and staying closed");
}

void setupRoofServos() {
    Serial.println("Initializing roof servos...");

    if (roofServo.attached()) {
        roofServo.detach();
        delay(100);
    }
    if (roofServo2.attached()) {
        roofServo2.detach();
        delay(100);
    }

    roofServo.setPeriodHertz(50);
    roofServo2.setPeriodHertz(50);

    bool success1 = roofServo.attach(SERVO_PIN, 500, 2500);
    bool success2 = roofServo2.attach(SERVO_PIN2, 500, 2500);

    if (!success1) {
        Serial.println("ERROR: Failed to attach first servo!");
    }
    if (!success2) {
        Serial.println("ERROR: Failed to attach second servo!");
    }

    Serial.println("Servos attached successfully");

    roofServo.write(SERVO_CLOSED_ANGLE);
    roofServo2.write(SERVO2_CLOSED_ANGLE);
    roofsOpen = false;
    delay(300);

    Serial.println("Servo initialization complete - Roof is CLOSED");
    Serial.printf("Servo1 closed position: %d°, Servo2 closed position: %d°\n",
        SERVO_CLOSED_ANGLE, SERVO2_CLOSED_ANGLE);
    Serial.printf("Servo1 open position: %d°, Servo2 open position: %d°\n",

```

```

        SERVO_OPEN_ANGLE, SERVO2_OPEN_ANGLE);
    }

void checkAndControlRoof() {

    bool currentRain = (digitalRead(RAIN_SENSOR_PIN) == LOW);

    Serial.print("Rain sensor: ");
    Serial.println(currentRain ? "RAIN DETECTED" : "No rain");

    Blynk.virtualWrite(VPIN_RAIN, currentRain ? "Rain Detected" : "No Rain");

    if (currentRain && roofsOpen) {
        Serial.println("Rain detected! Closing roof automatically...");
        CloseRoof();
        Blynk.logEvent("rain_alert", "Rain detected! Roof closed.");
    }
}

BLYNK_WRITE(V_FAN1_CONTROL) {
    int v = param.asInt();
    if (v == 2) {
        manualFan1 = false;
    } else {
        manualFan1 = true;
        digitalWrite(FAN1, v);
    }
}

BLYNK_WRITE(V_FAN2_CONTROL) {
    int v = param.asInt();
    if (v == 2) {
        manualFan2 = false;
    } else {
        manualFan2 = true;
        digitalWrite(FAN2, v);
    }
}

```

```

BLYNK_WRITE(V_FAN3_CONTROL) {
  int v = param.asInt();
  if (v == 2) {
    manualFan3 = false;
  } else {
    manualFan3 = true;
    digitalWrite(FAN3, v);
  }
}

```

```

BLYNK_WRITE(V_LED_CONTROL) {
  int v = param.asInt();
  if (v == 2) {
    manualLED = false;
  } else {
    manualLED = true;
    digitalWrite(LED1_PIN, v);
  }
}

```

```

BLYNK_WRITE(V_BUZZER_CONTROL) {
  int v = param.asInt();
  if (v == 2) {
    manualBuzzer = false;
  } else {
    manualBuzzer = true;
    digitalWrite(BUZZER, v);
  }
}

```

```

BLYNK_WRITE(V_ROOF_OPEN) {
  int value = param.asInt();
  if (value == 1) {
    Serial.println("Manual OPEN button pressed in Blynk");
    OpenRoof();
  }
}

```

```

BLYNK_WRITE(V_ROOF_CLOSE) {
  int value = param.asInt();
  if (value == 1) {
    Serial.println("Manual CLOSE button pressed in Blynk");
    CloseRoof();
  }
}

```

```

}

float readTemperature() {
  int raw = analogRead(TEMP_PIN);
  float voltage = raw * (3.3f / 4095.0f);
  temperatureC = voltage * 100.0f;

  Serial.printf("Temperature: %.1f °C\n", temperatureC);

  if (temperatureC > 35 && !tempAlertSent) {
    Blynk.logEvent("temp_alert", "High temperature detected!");
    tempAlertSent = true;
  }
  if (temperatureC <= 35) tempAlertSent = false;

  if (!manualFan1 && !manualFan2 && !manualFan3) {
    if (temperatureC < 20) {
      digitalWrite(FAN1, LOW); digitalWrite(FAN2, LOW); digitalWrite(FAN3, LOW);
    } else if (temperatureC < 25) {
      digitalWrite(FAN1, HIGH); digitalWrite(FAN2, LOW); digitalWrite(FAN3, LOW);
    } else if (temperatureC < 30) {
      digitalWrite(FAN1, HIGH); digitalWrite(FAN2, HIGH); digitalWrite(FAN3, LOW);
    } else {
      digitalWrite(FAN1, HIGH); digitalWrite(FAN2, HIGH); digitalWrite(FAN3, HIGH);
    }
  }
  return temperatureC;
}

int readLightLevel() {
  lightValue = analogRead(LDR_PIN);
  if (!manualLED) {
    digitalWrite(LED1_PIN, (lightValue > 3000) ? LOW : HIGH);
  }
  return lightValue;
}

bool readMotion() {
  motion = digitalRead(PIR_PIN);
  if (!manualBuzzer) {
    digitalWrite(BUZZER, motion ? HIGH : LOW);
  }
  return motion;
}

```



```

void sendDataToInfluxDB() {
  if (WiFi.status() != WL_CONNECTED) return;

  bool rain = (digitalRead(RAIN_SENSOR_PIN) == LOW);

  animalData.clearFields();
  animalData.addField("temperature", temperatureC);
  animalData.addField("light", lightValue);
  animalData.addField("motion", motion ? 1 : 0);
  animalData.addField("rain", rain ? 1 : 0);
  animalData.addField("roof_state", roofsOpen ? 1 : 0);
  animalData.addField("fan1", digitalRead(FAN1));
  animalData.addField("fan2", digitalRead(FAN2));
  animalData.addField("fan3", digitalRead(FAN3));
  animalData.addField("buzzer", digitalRead(BUZZER));
  animalData.addField("led", digitalRead(LED1_PIN));

  if (!influxClient.writePoint(animalData)) {
    Serial.println("InfluxDB write failed");
  }
}

```

```

unsigned long lastGpsSend = 0;
unsigned long lastGpsDebug = 0;
bool gpsTested = false;
unsigned long gpsTestTime = 0;

```

```

void sendGpsToBlynk() {

  while (gpsSerial.available() > 0) {
    char c = gpsSerial.read();
    gps.encode(c);
  }
}

```

```

unsigned long now = millis();

```

```

if (now - lastGpsDebug >= 2000) {
  lastGpsDebug = now;
}

```

```

int sats = gps.satellites.value();

Blynk.virtualWrite(V_GPS_SATS, sats);

if (gps.location.isValid()) {
  Blynk.virtualWrite(V_GPS_STATUS, "FIXED");
  String debugMsg = "GPS FIXED! Sats: " + String(sats);
  Blynk.virtualWrite(V_GPS_DEBUG, debugMsg);
  Blynk.virtualWrite(TEST, "GPS: Fixed");
} else {
  Blynk.virtualWrite(V_GPS_STATUS, "NO FIX");
  String debugMsg = "No fix | Sats: " + String(sats);
  Blynk.virtualWrite(V_GPS_DEBUG, debugMsg);
  Blynk.virtualWrite(TEST, "GPS: Waiting...");
}
}

if (now - lastGpsSend >= 5000) {
  lastGpsSend = now;

  if (gps.location.isValid()) {
    double lat = gps.location.lat();
    double lng = gps.location.lng();
    int sats = gps.satellites.value();

    Blynk.virtualWrite(V8,
      gps.location.lat(),
      gps.location.lng());

    String coords = "Lat: " + String(lat, 6) + "\nLon: " + String(lng, 6);
    Blynk.virtualWrite(V_GPS_COORD, coords);

    String quickGPS = String(lat, 4) + ", " + String(lng, 4);
    Blynk.virtualWrite(TEST, quickGPS);

  } else {
    Blynk.virtualWrite(V_GPS_COORD, "Waiting for GPS fix...");
  }
}
}

```

```

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n\n=== IoT Animal Housing System Starting ===");

  pinMode(TEMP_PIN,    INPUT);
  pinMode(LDR_PIN,     INPUT);
  pinMode(PIR_PIN,     INPUT);
  pinMode(RAIN_SENSOR_PIN, INPUT);
  pinMode(LED1_PIN,    OUTPUT);
  pinMode(FAN1,        OUTPUT);
  pinMode(FAN2,        OUTPUT);
  pinMode(FAN3,        OUTPUT);
  pinMode(BUZZER,      OUTPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(FAN1,    LOW);
  digitalWrite(FAN2,    LOW);
  digitalWrite(FAN3,    LOW);
  digitalWrite(BUZZER,  LOW);

  setupRoofServos();

  gpsSerial.begin(GPS_BAUD, SERIAL_8N1, GPS_RX, GPS_TX);

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  timeSync(TZ_INFO, "pool.ntp.org", "time.nis.gov");

  if (influxClient.validateConnection()) {
    Serial.println("InfluxDB connected");
  }

  animalData.addTag("device", "esp32_env");

  Serial.println("Setup complete - Roof is CLOSED");
}

```

```

    Serial.println("Press OPEN in Blynk to test\n");
}

unsigned long lastRoofCheck = 0;
unsigned long lastSensorUpdate = 0;
unsigned long lastInfluxSend = 0;
unsigned long lastGPSUpdate = 0;
unsigned long lastGPSDebugSend = 0;

void loop() {
    Blynk.run();

    unsigned long now = millis();

    if (!gpsTested && now > 10000) {
        gpsTested = true;
        int avail = gpsSerial.available();
        if (avail > 0) {
            Blynk.virtualWrite(V_GPS_DEBUG, "GPS Hardware OK - Receiving data");
        } else {
            Blynk.virtualWrite(V_GPS_DEBUG, "ERROR: No GPS data! Check wiring");
        }
    }
}

sendGpsToBlynk();

if (now - lastRoofCheck >= 1000) {
    lastRoofCheck = now;
    checkAndControlRoof();
}

if (now - lastSensorUpdate >= 2000) {
    lastSensorUpdate = now;

    temperatureC = readTemperature();
    lightValue = readLightLevel();
    motion = readMotion();

    Blynk.virtualWrite(V_TEMP, temperatureC);
    Blynk.virtualWrite(V_LIGHT, lightValue);
    Blynk.virtualWrite(V_MOTION, motion ? "Motion Detected" : "No Motion");
}

```

```
}  
  
if (now - lastInfluxSend >= 10000) {  
    lastInfluxSend = now;  
    sendDataToInfluxDB();  
}  
  
delay(10);  
}
```

ESP32 Code 2:

```
#define BLYNK_PRINT Serial
#define BLYNK_TEMPLATE_ID "TMPL6YKdWBVUb"
#define BLYNK_TEMPLATE_NAME "Animal Housing System"
#define BLYNK_AUTH_TOKEN "rcG8ISCgMqZmGT3iAGoHuSmgVkUag-TI"
#define INFLUXDB_URL "https://us-east-1-1.aws.cloud2.influxdata.com"
#define INFLUXDB_ORG "animal housing"
#define INFLUXDB_BUCKET "animal_housing"
#define INFLUXDB_TOKEN
"52hrfzPRKW_4T5_wN4WXoA4N6lXGLfGzmoHiB09c1Gud4HEhe0r3ZXNyqb9joYp8dwCjTwF
Wr3W56HZteziTAw=="
```

```
#define TZ_INFO "EET-2EEST,M3.5.5/0,M10.5.5/1"
```

```
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Arduino.h>
#include <Keypad.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>
#include <WiFiClientSecure.h>
#include <InfluxDbClient.h>
#include <InfluxDbCloud.h>
#include <TinyGPS++.h>
```

```
InfluxDBClient influxClient(
  INFLUXDB_URL,
  INFLUXDB_ORG,
  INFLUXDB_BUCKET,
  INFLUXDB_TOKEN,
  InfluxDbCloud2CACert
);
```

```
Point securityData("security_system");
```

```
char ssid[] = "HUAWEI-2.4G-jNmV";
char pass[] = "96JwMPDs";
```

```

#define SDA_PIN 8
#define SCL_PIN 9

#define WATER_LEVEL_PIN    10
#define WATER_PUMP_PIN     21

#define ROW1 38
#define ROW2 39
#define ROW3 40
#define ROW4 41

#define COL1 42
#define COL2 45
#define COL3 48
#define SERVO_PIN 13

#define IN1 4
#define IN2 5
#define IN3 7
#define IN4 6
#define ENA 16
#define ENB 17

#define PWM_FREQ 1000
#define PWM_RESOLUTION 8
#define CHANNEL_A 0
#define CHANNEL_B 1

#define VPIN_CHANGE_POSITION V17
#define VPIN_MOTOR_STATUS   V13
#define VPIN_GENERATED_CODE V20
#define VPIN_ACCESS_STATUS  V21
#define VPIN_DOOR_CONTROL V5
#define VPIN_WATER_LEVEL    V19
#define VPIN_WATER_PUMP_BTN V18

LiquidCrystal_I2C lcd(0x27, 16, 2);

Servo lockServo;
#define SERVO_LOCKED_POS 0
#define SERVO_OPEN_POS  90

```

```

const byte ROWS = 4;
const byte COLS = 3;

char keys[ROWS][COLS] = {
  {'1','2','3'},
  {'4','5','6'},
  {'7','8','9'},
  {'*','0','#'}
};

byte rowPins[ROWS] = {ROW1, ROW2, ROW3, ROW4};
byte colPins[COLS] = {COL1, COL2, COL3};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

String generatedCode = "";
String enteredCode = "";
bool enteringMode = false;
int speedValue = 180;
int doorStatus = 0;
int motorStatus = 0;
int accessStatus = 0;
unsigned long lastInfluxSend = 0;

void stopMotors() {
  ledcWrite(CHANNEL_A, 0);
  ledcWrite(CHANNEL_B, 0);

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);

  motorStatus = 0;
  Blynk.virtualWrite(VPIN_MOTOR_STATUS, "Stopped");
}

void moveForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  digitalWrite(IN3, HIGH);

```



```

digitalWrite(IN4, LOW);

ledcWrite(CHANNEL_A, 255);
ledcWrite(CHANNEL_B, 255);

motorStatus = 1;
Blynk.virtualWrite(VPIN_MOTOR_STATUS, "Moving Forward");
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  motorStatus = 1;
  ledcWrite(CHANNEL_A, 255);
  ledcWrite(CHANNEL_B, 255);

  Blynk.virtualWrite(VPIN_MOTOR_STATUS, "Turning Right");
}

void Change_Positions() {
  turnRight();
  delay(4000);

  stopMotors();
  delay(1000);

  moveForward();
  delay(3000);

  stopMotors();
}

String generate4DigitCode() {
  String code = "";
  for (int i = 0; i < 4; i++) {
    code += String(random(0, 10));
  }
  return code;
}

void showReadyScreen() {
  lcd.clear();
  lcd.setCursor(0, 0);

```

```

    lcd.print("Press * to start");
}

void resetSystem() {
    generatedCode = "";
    enteredCode = "";
    enteringMode = false;

    Blynk.virtualWrite(VPIN_GENERATED_CODE, "");
    Blynk.virtualWrite(VPIN_ACCESS_STATUS, "Waiting");

    showReadyScreen();
}

void updateLCDInput() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Enter Code:");
    lcd.setCursor(0, 1);
    lcd.print(enteredCode);
}

void startNewSession() {
    generatedCode = generate4DigitCode();
    enteredCode = "";
    enteringMode = true;

    Serial.print("Generated Code: ");
    Serial.println(generatedCode);

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Enter Code:");

    Blynk.virtualWrite(VPIN_GENERATED_CODE, generatedCode);
    Blynk.virtualWrite(VPIN_ACCESS_STATUS, "Code Sent");
}

void SERVO_CONTROL() {
    lockServo.write(SERVO_OPEN_POS);
    delay(2000);

    lockServo.write(SERVO_LOCKED_POS);
}

```

```

    delay(2000);
}

void Access_granted() {
    Serial.println("Access Granted");
    Blynk.virtualWrite(VPIN_ACCESS_STATUS, "Correct");
    accessStatus = 1;
    doorStatus = 1;
    lcd.clear();
    lcd.setCursor(0, 0);

    SERVO_CONTROL();

    delay(1000);
    resetSystem();
}

void Access_denied() {
    Serial.println("Access Denied");
    Blynk.virtualWrite(VPIN_ACCESS_STATUS, "Wrong");
    accessStatus = 0;

    enteredCode = "";

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Wrong! Try again");
    delay(1500);

    lcd.clear();
    lcd.setCursor(0, 0);

    lcd.print("Enter Code:");
}

bool manualPumpControl = false;
unsigned long lastWaterCheck = 0;

String lastWaterMessage = "";

void pumpOn() {
    digitalWrite(WATER_PUMP_PIN, LOW);
}

```

```

void pumpOff() {
  digitalWrite(WATER_PUMP_PIN, HIGH);
}

int lastWaterState = -1;

void checkWaterLevel() {
  int waterState = digitalRead(WATER_LEVEL_PIN);
  String currentMessage;

  if (waterState != lastWaterState) {

    if (waterState == LOW) {
      currentMessage = "Water Detected";

      Serial.println("Water Level: Water Detected");
      Blynk.virtualWrite(VPIN_WATER_LEVEL, currentMessage);

      if (!manualPumpControl) {
        pumpOff();
        Blynk.virtualWrite(VPIN_WATER_PUMP_BTN, 0);
      }

    } else {
      currentMessage = "No Water";

      Serial.println("Water Level: No Water");
      Blynk.virtualWrite(VPIN_WATER_LEVEL, currentMessage);

      if (!manualPumpControl) {
        pumpOn();
        Blynk.virtualWrite(VPIN_WATER_PUMP_BTN, 1);
      }
    }

    lastWaterState = waterState;
  }
}

BLYNK_WRITE(VPIN_CHANGE_POSITION) {
  int buttonState = param.asInt();

  if (buttonState == 1) {

```

```

    Change_Positions();
    Blynk.virtualWrite(VPIN_CHANGE_POSITION, 0);
  }
}
BLYNK_WRITE(VPIN_WATER_PUMP_BTN) {
  int state = param.asInt();

  manualPumpControl = true;

  if (state == 1) {
    pumpOn();
  } else {
    pumpOff();
  }
}
BLYNK_WRITE(VPIN_DOOR_CONTROL) {
  int state = param.asInt();

  if (state == 1) {
    lockServo.write(SERVO_OPEN_POS);
    doorStatus = 1;
    Serial.println("Door Opened");
  } else {
    lockServo.write(SERVO_LOCKED_POS);
    doorStatus = 0;
    Serial.println("Door Closed");
  }
}
void sendDataToInfluxDB() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("InfluxDB: WiFi not connected");
    return;
  }

  int waterState = digitalRead(WATER_LEVEL_PIN);
  int pumpState = digitalRead(WATER_PUMP_PIN) == LOW ? 1 : 0;

  securityData.clearFields();

  securityData.addField("water_level", waterState);
  securityData.addField("pump", pumpState);
  securityData.addField("door", doorStatus);
  securityData.addField("motor", motorStatus);
  securityData.addField("access", accessStatus);
}

```

```

    if (influxClient.writePoint(securityData)) {
        Serial.println("InfluxDB write successful");
    } else {
        Serial.print("InfluxDB write failed: ");
        Serial.println(influxClient.getLastErrorMessage());
    }
}

void setup() {
    Serial.begin(115200);
    randomSeed(micros());

    Wire.begin(SDA_PIN, SCL_PIN);
    lcd.init();
    lcd.backlight();

    pinMode(WATER_LEVEL_PIN, INPUT);
    pinMode(WATER_PUMP_PIN, OUTPUT);
    digitalWrite(WATER_PUMP_PIN, HIGH);

    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    pinMode(ENA, OUTPUT);
    pinMode(ENB, OUTPUT);

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    ledcSetup(CHANNEL_A, PWM_FREQ, PWM_RESOLUTION);
    ledcSetup(CHANNEL_B, PWM_FREQ, PWM_RESOLUTION);
    ledcAttachPin(ENA, CHANNEL_A);
    ledcAttachPin(ENB, CHANNEL_B);

    stopMotors();

    lockServo.setPeriodHertz(50);
    lockServo.attach(SERVO_PIN, 500, 2400);
    lockServo.write(SERVO_LOCKED_POS);

```

```

    pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);
pinMode(ENA, OUTPUT);
pinMode(ENB, OUTPUT);

digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);

ledcSetup(CHANNEL_A, 5000, 8);
ledcSetup(CHANNEL_B, 5000, 8);

ledcAttachPin(ENA, CHANNEL_A);
ledcAttachPin(ENB, CHANNEL_B);

stopMotors();
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

showReadyScreen();
Blynk.virtualWrite(VPIN_GENERATED_CODE, "");
Blynk.virtualWrite(VPIN_ACCESS_STATUS, "Waiting");
Blynk.virtualWrite(VPIN_MOTOR_STATUS, "Stopped");
checkWaterLevel();
Blynk.virtualWrite(VPIN_WATER_PUMP_BTN, 0);

timeSync(TZ_INFO, "pool.ntp.org", "time.nis.gov");

if (influxClient.validateConnection()) {
  Serial.print("Connected to InfluxDB: ");
  Serial.println(influxClient.getServerUrl());
} else {
  Serial.print("InfluxDB connection failed: ");
  Serial.println(influxClient.getLastErrorMessage());
}
}

void loop() {
  Blynk.run();

  if (!manualPumpControl && millis() - lastWaterCheck >= 1000) {
    lastWaterCheck = millis();
  }
}

```

```

    checkWaterLevel();
}

if (millis() - lastInfluxSend >= 10000) {
    lastInfluxSend = millis();
    sendDataToInfluxDB();
}

char key = keypad.getKey();

if (!key) {
    return;
}

Serial.print("Pressed: ");
Serial.println(key);

if (!enteringMode) {
    if (key == '*') {
        startNewSession();
    }
    return;
}

if (key >= '0' && key <= '9') {
    if (enteredCode.length() < 4) {
        enteredCode += key;
        updateLCDInput();
    }
}

else if (key == '#') {
    if (enteredCode == generatedCode) {
        Access_granted();
    } else {
        Access_denied();
    }
}

}

```